



KTO KARATAY
ÜNİVERSİTESİ

KTO KARATAY UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING

PROJE RAPORU

Ara Rapor

İsim ve Soyisim : Yasin Derya Bilgin
Bölüm : Bilgisayar Mühendisliği
Proje İsmi : RIMA - 2D Roguelite Aksiyon Oyunu
Tarih : Mayıs 2026

İçindekiler

Özet	3
1. Giriş.....	4
2. Proje Tanımı.....	5
2.1 Kapsam Dışı Unsurlar.....	5
2.2 Kapsam ve Temel Tanım.....	5
3. Sistem Tasarımı	6
3.1 Temel Oyun Döngüsü	6
3.2 Sınıf Sistemi	6
3.3 Yetenek Seçimi ve Kademe Sistemi	6
3.4 Harita ve Oda Sistemi	7
3.5 Kontrol ve Yön Sistemi.....	7
4. Yapılan Çalışmalar	8
4.1 Karakter Görsel Kimliği	8
4.2 Unity Entegrasyonu ve C# Altyapısı.....	9
4.3 Animasyon Üretim Kararı	9
4.4 Test ve Doğrulama.....	10
4.5 Geliştirme Sürecinde Agentic AI Kullanımı	10
5. Karşılaşılan Zorluklar.....	11
6. Sonraki Adımlar	12
7. Sonuç	13

Özet

RIMA, Unity 6 ile geliştirilen top-down 2D bir roguelite aksiyon oyunudur. Her koşuda oyuncu 10 sınıftan birini seçerek odalarda ilerler; temizlenen her oda sonrasında yetenek seçimiyle karakterini güçlendirir. Act 1 bossu yenildiğinde oyun bir kırılma noktasına ulaşır: oyuncuya ikinci bir sınıf sunulur ve iki sınıfın yetenek havuzları birleşerek sinerjik bir build oluşturulur. Act 2 bossu yenildiğinde ise iki sınıfın ortak ultimate yeteneği açılır.

Yetenek büyümesi üç katmanlı bir model üzerine kuruludur: Common ve Rare upgradeler temel istatistikleri düzenlerken, Epic seviyesinde yeteneğe yeni bir mekanik eklenir; Legendary seviyesinde ise yeteneğin davranış biçimi kökten değişebilir. Sınıflar arası sinerjiler tag sistemiyle okunabilir hale getirilir: Bleed, Shock, Mark, Curse gibi mekanik etiketler hangi sınıf kombinasyonlarının birbirini güçlendireceğini belirler. Bu yapı oyuncuya "daha fazla hasar" yerine "build'imı yönlendiriyorum" hissi verir.

Bu ara rapor, Faz 1 kapsamında tamamlanan sistem tasarımı, karakter görsel kimliği üretimi, Unity altyapısı, yön ve animasyon sistemi ile test çalışmalarını özetlemektedir.

1. Giriş

Roguelite türü, her oyun oturumunda farklı bir deneyim sunması ve kısa sürede tatmin edici bir ilerleme hissi oluşturmaları nedeniyle bağımsız oyun geliştirme alanında güçlü bir yere sahiptir. Ancak tür içindeki birçok oyunda build derinliği ile okunabilirlik arasında denge kurmak zordur. RIMA bu iki uç arasında dengeli bir yapı kurmayı amaçlamaktadır.

Projenin temel tasarım problemi, MMORPG türünde görülen güçlü ve sinerjik karakter yapısı kurma hissini, başı ve sonu olan tek bir roguelite koşusu içinde okunabilir ve yönetilebilir hale getirmektir. Roguelite türünde koşu süresi oyuncu deneyimine ve build seçimlerine göre büyük farklılıklar gösterir; bu nedenle tasarım sabit bir süre hedefi yerine her koşunun üç act ve iki boss ile yapılandırılmış tam bir ilerleme yayı sunması üzerine kurulmuştur. Bu hedef doğrultusunda sınıf seçimi, oda temizleme, yetenek taslağı, sınıf kırılma noktası ve çift sınıf sinerjileri tek bir ana döngü içinde birleştirilmiştir.

Tasarım kararları, ilgili türlerdeki oyunların aktif olarak oynanarak gözlemlenmesi ve oyuncu topluluklarının bu oyunlarda hangi sorunlarla karşılaştığı ile bu sorunlara nasıl çözümler üretildiğinin incelenmesi üzerine kurulmuştur. Mevcut çözümler başlangıç noktası olarak alınmış; ardından RIMA'nın kendi tasarım öncelikleri doğrultusunda alternatif yaklaşımlar geliştirilmiştir.

Referans Oyun	Projeye Aktarılan Tasarım Kararı
Hades	Oda tabanlı hızlı aksiyon akışı; saldırı yönü ile hareket yönünün ayrıştırılması
Slay the Spire	Oda sonrası üç seçenekli karar sunumu ve risk-ödül rotası
Guild Wars 1	Çift sınıf felsefesi ve sınıf havuzları arasında sinerji kurma
Enter the Gungeon	Top-down 2D aksiyon okunabilirliği ve mermi/savaş yoğunluğu
Children of Morta	Büyük karakter sprite estetiği ve sınıf kimliği hissi
Lost Ark / WoW / FFXIV	Sınıf kimliği, ayrı kaynak çubuğu ve rol farkı
Last Epoch	Skill specialization katman modeli; upgrade'lerin yalnızca sayısal güç değil davranış değişikliği de sunması ve tag tabanlı sınıf sinerjisi

2. Proje Tanımı

2.1 Kapsam Dışı Unsurlar

- Metroidvania değildir; sabit ve tamamen keşfedilebilir bir harita yerine her koşu prosedürel oda dizilimlerinden oluşur.
- Sıra tabanlı strateji oyunu değildir; savaş sistemi gerçek zamanlı ve aksiyon odaklıdır.
- Tam kapsamlı bir deckbuilder değildir; kart biriktirme yerine aktif yetenek slotları, yetenek geliştirme ve oda sonrası seçim sistemi kullanılır.
- MMORPG değildir; çevrim içi bileşen, sınırsız seviye ilerlemesi veya kalıcı ekipman sistemi hedeflenmez.

2.2 Kapsam ve Temel Tanım

RIMA, yaklaşık 35 derecelik yüksek top-down perspektifte oynanan, 2D pixel art estetiğine sahip bir roguelite aksiyon oyunudur. Her koşunun başında oyuncu 10 sınıftan birini birincil sınıf olarak seçer. Odaları temizledikçe yetenek seçim sistemi aracılığıyla karakterini güçlendirir. Birinci boss yenildiğinde oyun bir kırılma noktasına ulaşır: oyuncuya rastgele iki ikincil sınıf önerilir ve seçilen sınıfın yetenek havuzu mevcut karakter yapısına eklenir.

Bileşen	Araç / Detay
Oyun Motoru	Unity 6 (6000.3.6f1), URP 17.3
Programlama Dili	C#
Karakter Sanatı	Pixel art; 128x128 final sprite standardı, PPU=64
Sprite Üretimi	PixelLab, Aseprite / Pixelorama destekli kalite kontrol
Hedef Platform	PC - Steam
Versiyon Kontrolü	Git yerel depo

3. Sistem Tasarımı

3.1 Temel Oyun Döngüsü

1. Hub alanında oyuncu 10 sınıftan birini birincil sınıf olarak seçer.
2. Act 1 içinde oyuncu 8-9 odada yalnızca birincil sınıf yetenekleriyle ilerler.
3. Her savaş odasından sonra üç seçeneqli yetenek seçimi sunulur; belirli aralıklarla Echo Imprint seçeneği açılır.
4. Act 1 boss yenildiğinde kırılma noktası tetiklenir.
5. Oyuncuya rastgele iki ikincil sınıf önerilir ve bunlardan biri seçilir.
6. Act 2 içinde birincil ve ikincil sınıf havuzlarından gelen yeteneklerle ilerlenir.
7. Act 2 boss yenildiğinde sınıflar arası ultimate yetenek açılır.
8. Act 3 ve final boss aşamasında oyuncu çift sınıflı karakter yapısıyla koşuyu tamamlamaya çalışır.

3.2 Sınıf Sistemi

Projede 10 oynanabilir sınıf tasarlanmıştır. Her sınıfın kendine özgü kaynak yönetim mekaniği, görsel kimliği ve 12 yetenekten oluşan bir havuzu bulunmaktadır.

Sınıf	Arketip	Kaynak Mekaniği
Warblade	Ağır yakın dövüş, kitle kontrol	Rage
Shadowblade	Hızlı yakın dövüş, kaçış	Energy
Ranger	Menzilli, kritik isabet odaklı	Focus
Ronin	Kontr-saldırı, savuşturma	Resolve
Gunslinger	Çok hedefli ateşli silah	Ammo
Brawler	Yakın mesafe, sendeletme	Stamina
Ravager	Yavaş ama yıkıcı darbeler	Wrath
Elementalist	Alan hasarı, element kombinasyonu	Mana
Hexer	DoT, debuff, zayıflatma	Hexes
Summoner	Çağırma, vekil yönetimi	Souls

3.3 Yetenek Seçimi ve Kademe Sistemi

Roguelite türünde oda sonrası seçim sunumunun oyuncuya "ne almak istiyorum" yerine "şu an hangi seçenek daha değerli" sorusunu sordurması, build kimliğinin kısa sürede netleşmesini sağlayan temel bir mekanik olarak gözlemlenmiştir. Slay the Spire bu dinamiği kart seçimiyle oluştururken, yetenek slotuna dayalı aksiyon oyunlarında aynı karar ağırlığını aktarmak daha zordur çünkü oyuncu yeteneği pasif kazanmak yerine aktif kullanmak zorundadır. RIMA'da yeni bir yetenek edinimi, mevcut bir yeteneğin kademesini yükseltme ve Echo Imprint seçenekleri bir arada sunularak her oda sonrasında farklı türde bir karar baskısı yaratılmaktadır.

Yetenek büyümesi Last Epoch'taki skill specialization anlayışından ilham alarak üç katmanlı bir model üzerine kurulmuştur. **Common / Rare** kademeler hasar, cooldown, alan gibi temel parametreleri düzenler. **Epic** kademe yeteneğe yeni bir yan mekanik ekler; örneğin bir geniş alan saldırısı Bleed durumu bırakmaya ya da tek hedefe odaklanarak boss hasarını artırmaya başlar. **Legendary** kademe ise yeteneğin davranış biçimini kökten değiştirebilir; ancak bu katman nadir tutularak sınıf kimliğinin bulanıklaşması önlenir.

Sınıflar arası sinerjilerin okunabilirliği için tag sistemi kullanılmaktadır. Her yetenek bir veya birden fazla mekanik etiket taşır: Bleed, Burn, Shock, Curse, Mark, DoT, Projectile, Area, Dash gibi. Çift sınıf kombinasyonlarının birbirini nasıl güçlendireceği bu etiketler üzerinden belirlenir; örneğin Warblade ve

Hexer kombinasyonunda Bleed, Curse ve DoT etiketli yetenekler sinerjik bir baskı build'i oluşturur. Bu yapı oyuncuya "build'imi rastgele şekillendiriyorum" değil, "build'imi yönlendiriyorum" hissi verir.

3.4 Harita ve Oda Sistemi

Harita tasarımında oyuncuya kısa vadeli rota planlaması sunan, ancak tüm koşu akışını baştan açık etmeyen katmanlı bir görünürlük yapısı kullanılmaktadır. Oyuncu mevcut katmandaki 5-7 odalık bağlantı bölümünü görebilir; ilerideki odalar kapalı kalır. Kapılar oda tipini simge ve renkle gösterir.

3.5 Kontrol ve Yön Sistemi

İzometrik top-down oyunlarda hareket yönü ile görsel yön arasındaki uyumsuzluk, özellikle saldırı isabeti ve animasyon kesme sorunları olarak kendini gösteren yaygın bir tasarım problemidir. Birçok oyun bu sorunu tam sekiz yönlü sprite seti üreterek çözmeye çalışmıştır; ancak bu yaklaşım üretim yükünü katlamakta ve silüet okunabilirliğini azaltmaktadır. Hades bu probleme farklı bir çözüm getirmiştir: hareketi, görsel yönü ve saldırı yönünü birbirinden bağımsız katmanlara ayırarak oyuncunun saldırı hedefini hareket yönünden bağımsız seçmesine olanak tanımıştır. Buna karşın Hades 1'de bu ayrışmanın oyunculara alışılmadık bir "tank dönüşü" izlenimi verdiği gözlemlenmiştir; Hades 2 ise bu sorunu yumuşak geçiş animasyonları ve eğilim blendleri ekleyerek gidermiştir.

RIMA bu gözlemlerden yola çıkarak farklı bir yaklaşım uygulamaktadır. Görsel yön, izometrik perspektife uygun dört çapraz yönle sınırlandırılmıştır: SE, NE, NW, SW. Sekiz yönlü sprite seti gereksinimine yol açmadan güçlü silüet okunabilirliği korunmaktadır. Hareket, görsel yön ve saldırı/skill yönü ayrı katmanlar olarak uygulanmıştır: SON YÖN modunda saldırılar karakterin son baktığı yöne gider; MOUSE modunda ise saldırı veya skill tetiklendiği anda karakter mouse imlecine döner ve saldırı o yöne uygulanır. Mouse hareketi tek başına yürüyüş yönünü değiştirmez; bu sayede sürekli mouse takibi olmaksızın Hades benzeri bir hedefleme hissi elde edilmektedir. Keskin yön kesmelerini yumuşatmak için görsel bir geçiş gecikmesi eklenmiş; saldırı ve skill kullanımı ise oynanışın tepkisel kalması için bu gecikmeyi her zaman aşmaktadır.

4. Yapılan Çalışmalar

4.1 Karakter Görsel Kimliği

Her sınıf için öncelikle bir referans sprite, yani anchor sprite üretilmiştir. Onaylanan anchor, sonraki rotasyon, idle, run ve saldırı animasyonlarının üzerine inşa edildiği temel görseldir. Mevcut durumda 10 sınıfın tamamı için anchor üretimi tamamlanmış ve Unity entegrasyonu için kullanılabilir hale getirilmiştir.

Sınıf Anchor Görselleri

				
Warblade	Shadowblade	Ranger	Ronin	Gunslinger

				
Brawler	Ravager	Elementalist	Hexer	Summoner

4.2 Unity Entegrasyonu ve C# Altyapısı

Bu fazda projenin savaş, oda akışı, draft sistemi, karakter seçimi ve animasyon altyapısı için çok sayıda C# betiği geliştirilmiştir. Ayrıca wave-1 karakterleri olan Warblade, Elementalist, Ranger ve Shadowblade için idle sprite ve animator controller entegrasyonu yapılmıştır.

Script	Açıklama
`PlayerController.cs`	Oyuncu hareket sistemi. WASD girişini işler, dash mekaniklerini yönetir; hareket yönü ile saldırı yönünü birbirinden bağımsız katmanlar olarak tutar.
`PlayerAnimator.cs`	Oyuncu animasyon kontrolcüsü. Girişe göre dört çapraz görsel yönden birini seçer, yön geçişlerini yumuşatır ve saldırı sırasında görsel yönü anında günceller.
`PlayerAttack.cs`	Temel saldırı sistemi. Üç vuruşluk combo zincirini, hitbox tetiklemesini ve VFX'ı yönetir; saldırı yönünü `SON YÖN` veya `MOUSE` moduna göre belirler.
`SkillBase.cs`	Tüm aktif yeteneklerin türediği temel sınıf. Yetenek kullanımından önce saldırı yönünü günceller; her yetenek sınıfı bu altyapıyı miras alır.
`SettingsMenuUI.cs`	Oyun içi ayarlar menüsü. ESC tuşuyla açılır ve kapanır; saldırı/yetenek yön modu gibi oynanış tercihlerini runtime'da değiştirmeye olanak tanır.
`CharacterSelectScreen.cs`	Sınıf seçim ekranı. Koşu başında oyuncuya 10 sınıfı sunar; seçim yapıldığında Animator controller'ı değiştirir ve sınıf yöneticisini bilgilendirir.
`DungeonGraph.cs`	Prosedürel oda grafiği. Her koşuda odaları ve aralarındaki bağlantıları üretir; harita yapısının veri modelini tutar.
`RuntimeRoomManager.cs`	Oda akış yöneticisi. Aktif odayı takip eder, oda temizlendiğinde geçiş mantığını çalıştırır ve draft sunumunu tetikler.
`DraftManager.cs`	Yetenek seçim sistemi. Oda sonrasında oyuncuya üç seçenek sunar: yeni yetenek, kademe yükseltme veya Echo Imprint.
`BossAI_PenitentSovereign.cs`	Act 1 boss yapay zekası. Boss davranış fazlarını, saldırı örüntülerini ve kırılma noktası tetiklemesini yönetir.

Geliştirilen scriptler, onaylanmış anchor ve rotasyon sprite referansları üzerine yazılmıştır; run animasyon sprite sheetleri henüz üretim aşamasındadır. Animasyonlar PixelLab ile üretildikçe Unity Animator bağlantıları kurulacak ve hareket ile savaş sistemi gerçek sprite setleriyle oyun içinde test edilecektir.

4.3 Animasyon Üretim Kararı

Aksiyon odaklı roguelite oyunlarda yürüyüş animasyonunun çok az kullanıldığı, oyuncunun büyük çoğunlukla koşar hızında hareket ettiği gözlemlenmiştir. Ayrıca yürüyüş ve koşu animasyonlarının ayrı üretimi sprite bütçesini ikiye katlamaktadır. Bu durumu göz önünde bulunduran bazı oyunlar yalnızca koşu animasyonu kullanmış; hareket hızını düşürerek yürüyüş simüle etmiştir. RIMA animasyon pipeline'i da run-first yaklaşımını benimsemiştir: walk animasyonu üretimi şimdilik kapsam dışıdır, ana hareket animasyonu run olacaktır. Üretim standardı dört çapraz yöndür: run_SE, run_NE, run_NW ve run_SW. Bu kararla sprite bütçesi korunmakta, sınıf kimliğini belirleyecek run animasyonuna odaklanılmaktadır.

4.4 Test ve Doğrulama

Unity Test Runner üzerinden EditMode ve PlayMode testleri yürütülmüştür. Son doğrulamalarda EditMode test paketi 128/128 başarılı sonuç vermiştir. Önceki PlayMode oda akışı testleri de 24/24 başarılı olarak kaydedilmiştir. Script validation kontrollerinde temel oynanış ve yön sistemi dosyalarında derleme hatası görülmemiştir.

4.5 Geliştirme Sürecinde Agentic AI Kullanımı

Bu proje solo geliştirme olarak yürütülmekle birlikte, geliştirme sürecinin tamamında agentic AI sistemleriyle eş zamanlı çalışılmıştır. Agentic AI, tek bir soru-cevap etkileşiminin ötesinde; görevi anlayan, adımlara bölen, araçları bağımsız olarak kullanan ve çıktılarını daha büyük bir iş akışına entegre eden otonom yapay zeka sistemleri anlamına gelmektedir.

Projede benimsenen yaklaşım orkestrasyon modeline dayanmaktadır. Ana ajan projenin genel durumunu takip eder, hangi işin hangi uzmanlık ajanına yönlendirileceğine karar verir ve çıktıları bütünleştirir. Bu model altında kod üretimi ve Unity entegrasyonu bir kod ajanına, sprite üretimi ve animasyon pipeline kararları bir görsel üretim ajanına, tasarım kararları ise bir tasarım ajanına devredilmiştir. Geliştirici bu süreçte teknik yürütücü yerine kararları veren ve iş akışını yönlendiren konumda çalışmıştır.

Günümüzde agentic AI sistemleri yazılım ve oyun geliştirme pratiğini köklü biçimde dönüştürmektedir. Statik kod tamamlama araçlarının yerini; bağlamı oturum boyunca koruyan, birden fazla dosyayı aynı anda düzenleyebilen, test çalıştıran ve uçtan uca görevleri tamamlayabilen ajan sistemleri almaktadır. RIMA'nın Faz 1 kapsamında 10 sınıf ve 120'den fazla skill tanımı, Unity C# altyapısı, animasyon pipeline kararları ve test süitinin paralel olarak ilerlemesi bu iş akışının doğrudan bir çıktısıdır. Solo geliştirici bağlamında agentic AI orkestrasyon modeli, geleneksel bir ekibin paralel çalışma kapasitesini tek bir iş akışında yeniden üretmektedir.

5. Karşılaşılan Zorluklar

Stil tutarlılığı: On farklı sınıf için pixel art üretiminde sınıf kimliğini korumak ile genel stil bütünlüğünü sağlamak sürekli bir denge gerektirmiştir.

Yön ve animasyon eşlemesi: İzometrik karakterlerde kuzey-güney-doğu-batı adlandırması ile görsel yön arasında tutarsızlık kolayca oluşabilmektedir; benzer projelerde sprite adlandırma hatalarının animasyon pipelineını bozduğu yaygın olarak görülmektedir. Bu projede de PixelLab rotasyon dosyaları ile oyun içi yön adları arasında tutarsızlık tespit edilmiştir. Çözüm olarak dosya adı yerine gerçek görsel içerik ve Unity animator bindingleri referans alınmış; her yön dosyası frame frame kontrol edilerek canonical adlandırma tutarlı hale getirilmiştir.

Kaynak sistemi çeşitliliği: 10 sınıfın farklı kaynak mekaniklerine sahip olması hem kod mimarisini hem de denge tasarımını zorlaştırmıştır.

Tek geliştirici kapasitesi: Projenin solo geliştirme olarak yürütülmesi nedeniyle sanat üretimi, sistem tasarımı, programlama ve test çalışmaları paralel ilerlemektedir.

6. Sonraki Adımlar

Öncelik	İş
1	Warblade `run_SE`, `run_NE`, `run_NW` ve `run_SW` animasyonlarının PixelLab ile üretilmesi
2	Üretilen run sprite sheetlerinin Unity import ayarlarının yapılması ve animator controllara bağlanması
3	Attack startup frame ve yön değişimi hissini saklayan ek animasyon polish çalışmaları
4	Elementalist, Ranger ve Shadowblade için aynı 4 diagonal run pipelineinin uygulanması
5	HandGlowVFX ve sınıf kaynak görsel efektlerinin prefab bağlantılarının tamamlanması
6	Tile/object üretimi ile test sahnesinin görsel arka planının güçlendirilmesi
7	Combat -> draft -> oda geçiş akışının tam oynanabilir Faz 1 prototipinde tekrar doğrulanması

7. Sonu

RIMA, ift sınıflı karakter geliştirme sistemi ile roguelite oda akışını birleştiren özgün bir oyun tasarımı hipotezini teknik ve sanatsal açıdan hayata geçirmeyi hedefleyen bir bilgisayar mühendisliği projesidir. Birinci faz kapsamında temel sistem mimarisi kurgulanmış, savaş ve oda akışı altyapısı geliştirilmiş, 10 sınıfın görsel kimliği oluşturulmuş ve Unity içinde karakter seçimi, idle animasyon entegrasyonu, ayar menüsü ve test altyapısı kurulmuştur.

Son güncellemelerle oyuncu yön sistemi izometrik oynanışa daha uygun hale getirilmiş, saldırı ve yetenek yönlendirmesi hareket yönünden ayrılmıştır. Bu sayede proje yalnızca görsel üretim değil, aynı zamanda gerçek zamanlı kontrol, animasyon seçimi, test edilebilir sistem mimarisi ve kullanıcı ayarları açısından da daha olgun bir prototip seviyesine yaklaşmıştır.

Bu belge Mayıs 2026 itibarıyla projenin Faz 1 ara durumunu yansıtmaktadır.